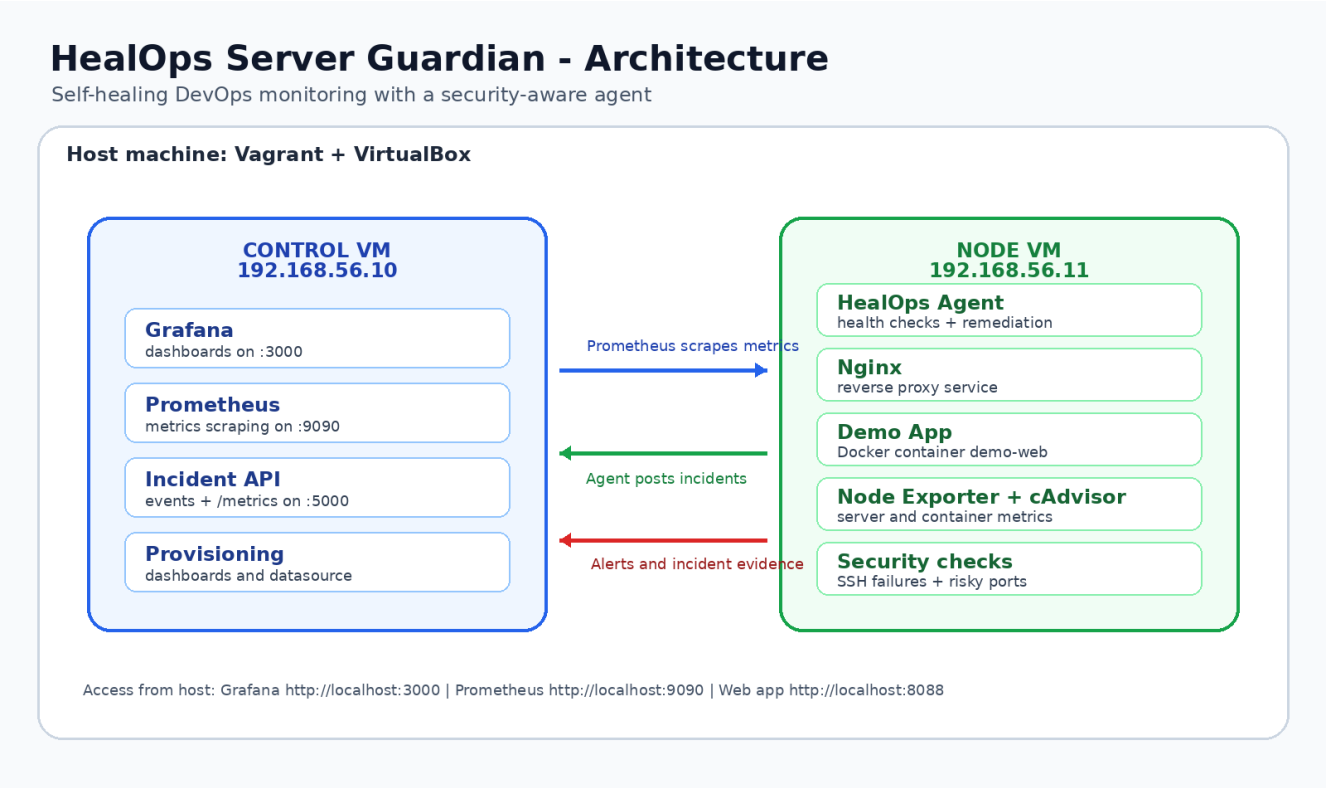
# HealOps Server Guardian

Infrastructure DevOps auto-reparatrice avec surveillance serveur et detection securite

Version projet - Francais



# 1. Resume executif

HealOps Server Guardian est un projet DevSecOps qui transforme un simple monitoring en une plateforme de self-healing. Le systeme surveille un serveur Linux complet: services systemd, conteneurs Docker, ressources, ports reseau et journaux SSH. Lorsqu'une panne ou un signal securite est detecte, l'agent declenche une action controlee: redemarrage de service, relance de conteneur, creation d'incident et alerte.

- Objectif DevOps: assurer la disponibilite et reduire le MTTR par l'automatisation.
- Objectif securite: detecter les signaux simples d'attaque comme les echecs SSH repetes ou les ports suspects.
- Objectif presentation: fournir une demo live claire, mesurable et professionnelle en deux jours.

# 2. Probleme traite

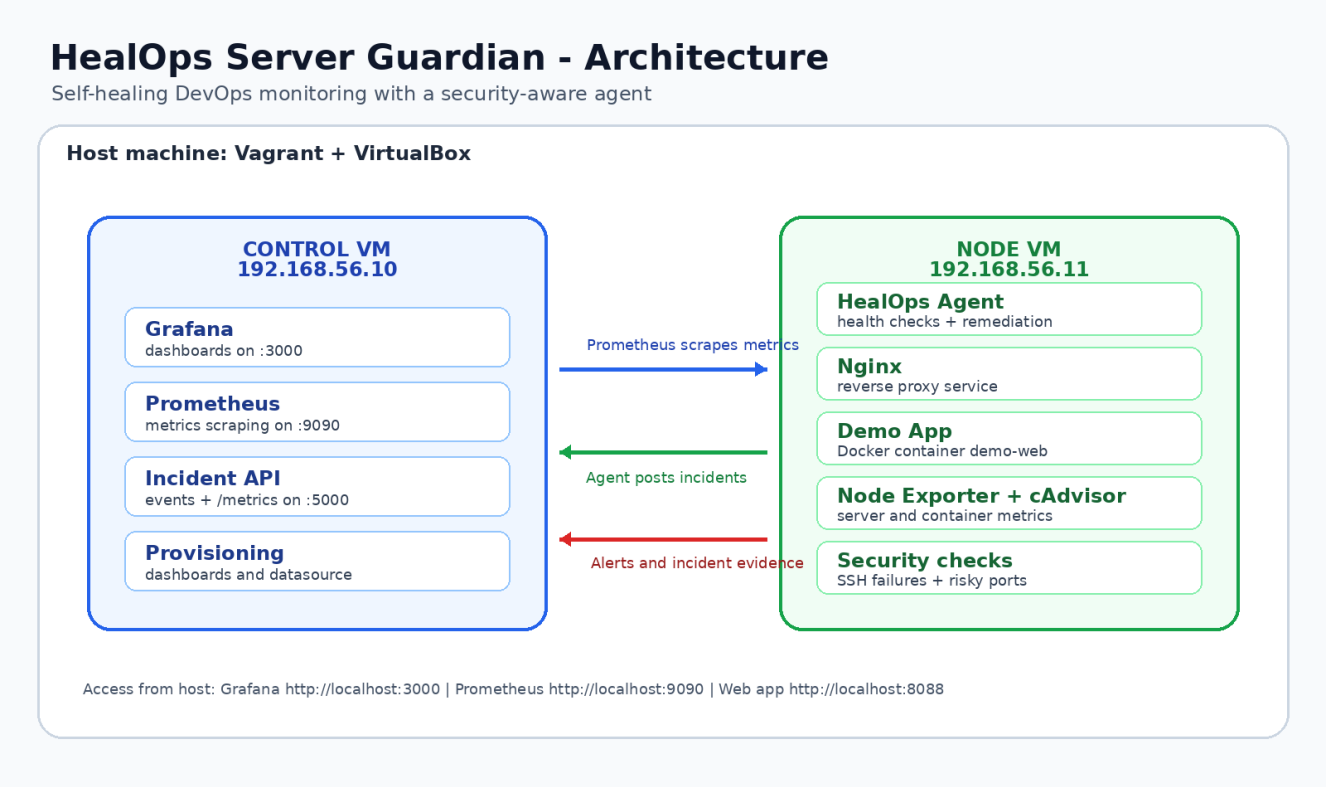
Dans une infrastructure reelle, une application peut etre disponible dans Docker, mais tomber a cause d'un reverse proxy arrete, d'un conteneur down, d'un disque presque plein ou d'activite suspecte sur SSH. Un projet classique affiche seulement une alerte. HealOps ajoute une etape plus avancee: l'auto-remediation.

- Detection: verifier l'etat du serveur, des services et des conteneurs.
- Decision: classifier l'incident selon son impact: medium, high ou critical.
- Remediation: appliquer uniquement les actions safe comme restart Nginx ou docker start.
- Observabilite: centraliser les metriques et incidents dans Prometheus et Grafana.

# 3. Objectifs du projet

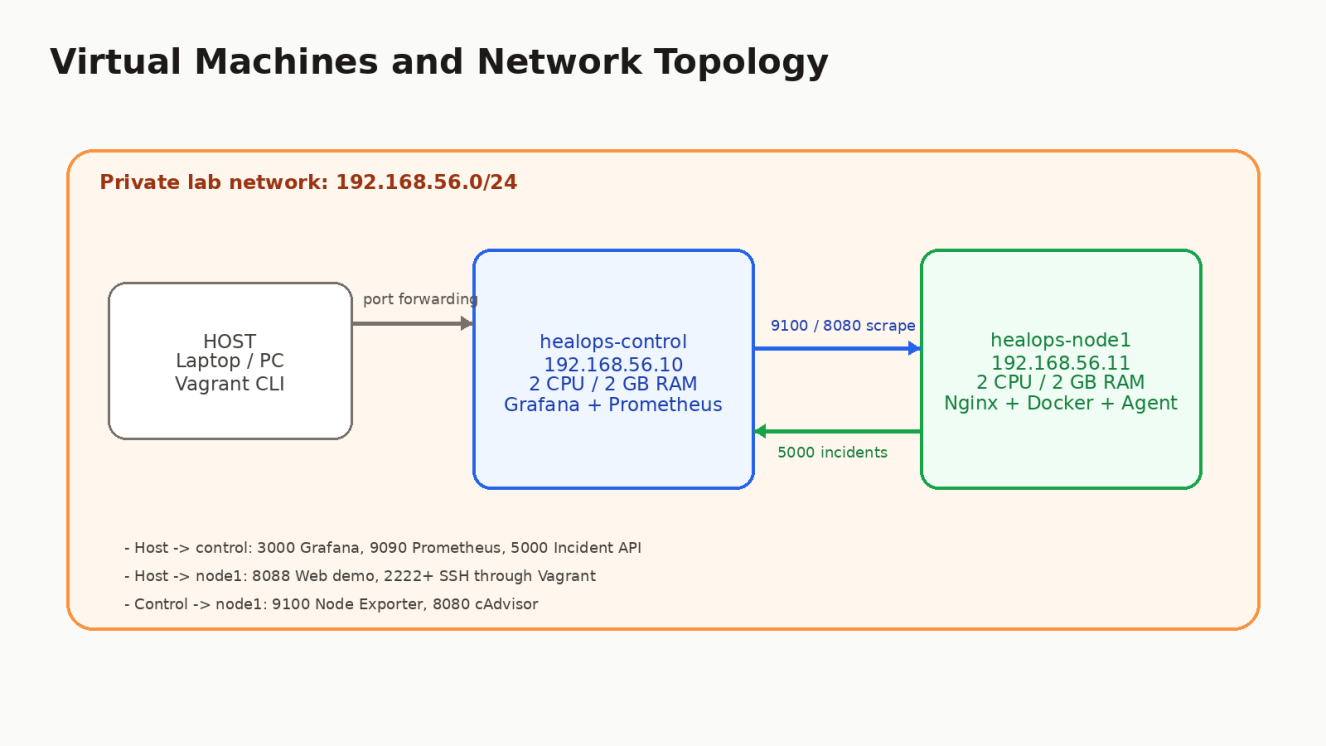
| Objectif | Description | Impact |
|---|---|---|
| Monitoring complet | CPU, RAM, disque, Nginx, Docker, ports, SSH | Vision serveur complete |
| Self-healing | Restart automatique des composants critiques | MTTR reduit |
| Security-aware | Detection brute force SSH et ports suspects | DevSecOps visible |
| Dashboard | Grafana + Prometheus | Demo claire |
| Automation | Vagrantfile + scripts de provisioning | Lab reproductible |

# 4. Architecture globale




L'architecture separe le role de controle et le role serveur applicatif. La VM control centralise Prometheus, Grafana et l'API d'incidents. La VM node1 execute l'application, le reverse proxy, les exporters et l'agent HealOps.

# 5. Machines virtuelles et reseau



| VM | Adresse | Role | Ports principaux | Ressources |
|---|---|---|---|---|
| control | 192.168.56.10 | Monitoring et centralisation | 3000 Grafana, 9090 Prometheus, 5000 API | 2 CPU, 2 GB RAM |
| node1 | 192.168.56.11 | Serveur applicatif protege | 80 Nginx, 9100 metrics, 8080 cAdvisor | 2 CPU, 2 GB RAM |

Le host accede aux services via port forwarding: Grafana sur localhost:3000, Prometheus sur localhost:9090, Incident API sur localhost:5000 et application demo sur localhost:8088.

## 6. Composants techniques

| Composant | Fonction |
|---|---|
| Vagrant | Creation automatique des deux VMs et lancement du provisioning |
| VirtualBox | Hyperviseur local pour executer le lab |
| Docker | Execution de l application demo, Node Exporter, cAdvisor, Prometheus, Grafana |
| Nginx | Reverse proxy devant le conteneur demo-web |
| Prometheus | Collecte des metriques serveur, conteneurs et incidents |
| Grafana | Visualisation CPU, RAM, disque, conteneurs et incidents |
| HealOps Agent | Detection de pannes, remediation et envoi d incidents |
| UFW / Fail2ban | Base de durcissement reseau et signal securite |

## 7. Fonctionnement de l agent HealOps

L'agent est installe comme service systemd sur la VM node1. Il tourne en boucle et execute des controles toutes les 15 secondes. Chaque incident est journalise localement, envoye a l'API control et expose ensuite a Prometheus.

| Controle | Condition | Action |
|---|---|---|
| Nginx | systemctl is-active nginx echoue | systemctl restart nginx |
| Docker app | demo-web n est pas running | docker start demo-web |
| CPU/RAM/Disk | Seuil eleve | Alerte et incident |
| SSH | Plusieurs Failed password depuis la meme IP | Alerte, blocage optionnel UFW |
| Ports suspects | Port 4444, 5555, 1337 ou 31337 en ecoute | Alerte securite |

## 8. Plan de realisation en deux jours

| Jour | Travail | Resultat attendu |
|---|---|---|
| Day 1 - Matin | Installer la structure projet, Vagrantfile, reseau prive et VMs | vagrant up cree control et node1 |
| Day 1 - Apres-midi | Installer Docker, Nginx, app demo, Prometheus, Grafana | Dashboard accessible et application demo online |
| Day 1 - Fin | Verifier les exporters et la collecte metrique | CPU/RAM/Disk visibles dans Grafana |
| Day 2 - Matin | Installer l agent HealOps et API incidents | Incidents envoyes au control |
| Day 2 - Apres-midi | Ajouter scenarios demo: stop Nginx, stop container, SSH failed, CPU stress | Demo live complete |
| Day 2 - Fin | Nettoyer README, captures, rapport et presentation | Projet presentable et reproductible |

# 9. Fichier d automatisation pret a utiliser

Le package contient un Vagrantfile et des scripts shell. La commande suivante cree les VMs, installe les dependances et prepare tout l'environnement de travail:

```
unzip healops_server_guardian_lab.zip
cd healops-server-guardian-lab
vagrant up
```

Le provisioning installe automatiquement Docker, Docker Compose plugin, Python, Nginx, UFW, Fail2ban, Prometheus, Grafana, Node Exporter, cAdvisor, l'application demo et les services HealOps.

# 10. Structure du package

```
healops-server-guardian-lab/
  Vagrantfile
  scripts/
    provision_common.sh
    provision_control.sh
    provision_node.sh
    requirements.txt
    demo_scenarios.sh
  healops/
    agent.py
    incident_api.py
  compose/
    control/
    node/
  README_FR.md
  README_EN.md
```

# 11. Scenarios de demonstration

## Scenario A - Panne Nginx

```
vagrant ssh node1
sudo systemctl stop nginx
sudo journalctl -u healops-agent -f
```

Resultat attendu: l'agent detecte le service down, redemarre Nginx, cree un incident high et l'envoie a la VM control.

## Scenario B - Conteneur Docker arrete

```
vagrant ssh node1
sudo docker stop demo-web
sudo docker ps --filter name=demo-web
```

Resultat attendu: le conteneur est relance automatiquement par l'agent.

## Scenario C - Brute force SSH simule

```
for i in {1..6}; do
  sudo logger -p authpriv.warning "sshd[12345]: Failed password for invalid user admin from
192.168.56.99 port 55${i} ssh2"
done
```

Resultat attendu: l'agent detecte plusieurs echecs SSH depuis la meme IP et cree un incident security-aware.

# 12. Ce qui rend le projet professionnel

- Il ne surveille pas seulement une application: il surveille le serveur complet.
- Il ne fait pas seulement des alertes: il applique une remediation controlee.
- Il montre DevOps clairement: VMs, Docker, Nginx, Prometheus, Grafana, systemd et automation.
- Il ajoute une couche securite sans complexite excessive: SSH failures, ports suspects, UFW et incidents.
- Il est reproductible grace au Vagrantfile et aux scripts de provisioning.

## 13. Limites et ameliorations possibles

| Limite actuelle | Amelioration future |
|---|---|
| Detection securite simple | Ajouter Falco ou Wazuh pour runtime detection avancee |
| Une seule node applicative | Ajouter node2 et simuler haute disponibilite |
| Blocage IP desactive par defaut | Mode production avec playbook de reponse valide |
| Pas de CI/CD inclus | Ajouter pipeline GitHub Actions pour deployer l app demo |
| Dashboard initial | Ajouter panels incident timeline et SLA uptime |

## 14. Phrase de presentation

"J ai concu une infrastructure DevSecOps auto-reparatrice qui surveille un serveur Linux complet, detecte les pannes et signaux securite, puis applique des actions de remediation avec observabilite en temps reel."